

```

1  (*
2                                     CS51 Lab 13
3                               Procedural Programming and Loops
4  *)
5
6  (*=====
7  Readings:
8
9      This lab builds on material from Chapter 16 of the textbook
10     <http://book.cs51.io>, which should be read before the lab session.
11
12 Objective:
13
14     This lab introduces and provides practice with:
15     - loops and procedural programming, and
16     - tail recursion as an alternate form of iteration.
17
18     It is structured in four parts:
19
20     1. Loops as a solution to stack space problems
21     2. Practice with procedural programming
22     3. Tail recursion as a solution to stack space problems
23     4. The Mario problem
24  =====*)
25
26
27  (*=====
28     Part 1: Loops as a solution to stack space problems
29
30     In Lab 2, you created recursive functions to find the lengths and sums
31     of lists. Below are examples of both:
32
33     let rec length (lst : int list) : int =
34         match lst with
35         | [] -> 0
36         | _hd :: tl -> 1 + length tl ;;
37
38     let rec sum (lst : int list) : int =
39         match lst with
40         | [] -> 0
41         | hd :: tl -> hd + sum tl ;;
42
43     As noted in the textbook, these functional recursive implementations
44     may run into stack overflow errors when called on exceptionally long
45     lists.
46
47     # sum (List.init 1_000_000 Fun.id) ;;
48     Stack overflow during evaluation (looping recursion?).
49
50     As computation proceeds, each recursive call of `length tl` or `sum
51     tl` is suspended, with the suspended computation stored on a stack,
52     until we reach the end of the list. At that point, the stack finally

```

```

53 unwinds and the expression is evaluated. If the number of calls grows
54 too large, we run out of room allocated to the stack of suspended
55 computations, and the computation fails with a `Stack_overflow`
56 exception.
57
58 Two ways of addressing this problem are
59
60 (i) using an explicit loop, or
61
62 (ii) using a tail-recursive function.
63
64 Let's try the loop approach first. (We'll return to the tail-recursion
65 approach in Part 3.) *)
66
67 (*.....
68 Exercise 1: Rewrite the functional recursive `sum` function from above
69 using a `while` loop.
70
71 For reference, here is the `length` function implemented using a
72 `while` loop, as in the reading:
73
74     let length_iter (lst : 'a list) : int =
75         let counter = ref 0 in          (* initialize the counter *)
76         let lstr = ref lst in          (* initialize the list *)
77         while !lstr <> [] do            (* while list not empty... *)
78             counter := succ !counter;   (* increment the counter *)
79             lstr := List.tl !lstr       (* drop element from list *)
80         done;
81         !counter ;;                     (* return the counter value *)
82
83 Note that both the counter for the loop and the list need to be
84 references. Otherwise, their values can't be changed and the loop
85 will never terminate.
86 .....*)
87
88 let sum_iter (lst : int list) : int =
89     failwith "sum_iter not implemented" ;;
90
91 (* Once you've implemented it successfully, you should be able to sum
92    over extremely long lists, for example,
93
94        # sum_iter (List.init 1_000_000 Fun.id) ;;
95        - : int = 499999500000
96    *)
97
98 (*=====
99 Part 2: Practice with procedural programming
100
101 You've implemented one list-processing function, `sum`, in a
102 procedural style. Time to get some more practice with this paradigm.
103
104 .....
105 Exercise 2: Reimplement another familiar function: reversing a
106

```

```

107 list. Write an iterative function `reverse` to reverse a list. (By the
108 way, this function is implemented tail-recursively in the `List`
109 module as `List.rev`. See
110 https://github.com/ocaml/ocaml/blob/trunk/stdlib/list.ml.)
111 .....*)
112
113 let reverse (lst : 'a list) : 'a list =
114   failwith "reverse not implemented" ;;
115
116 (*.....
117 Exercise 3: Write two non-recursive functions, `odds_while` and
118 `odds_for`, that use `while` and `for` loops, respectively, to return
119 a list of all positive odd numbers less than or equal to a given
120 `int`. (Don't worry about dealing with negative arguments.)
121
122 For example, we expect the following behavior:
123
124   # odds_while 10
125   - : int list = [1; 3; 5; 7; 9]
126   # odds_for 7
127   - : int list = [1; 3; 5; 7]
128 .....*)
129
130 let odds_while (limit : int) : int list =
131   failwith "odds_while not implemented" ;;
132
133 let odds_for (limit : int) : int list =
134   failwith "odds_for not implemented" ;;
135
136 (* The `prods` function calculates the product of the corresponding
137 elements of two integer lists. Your function should raise a `Failure`
138 exception when called on lists of differing length. You may remember
139 implementing a similar (though not identical) function in Lab 2. It
140 should work like this:
141
142   # prods [1; 2; 3] [1; 2; 3] ;;
143   -: int list = [1; 4; 9]
144
145 Here is a straightforward recursive version of the `prods` function.
146
147   let rec prods (xs : int list) (ys : int list) : int list =
148     match xs, ys with
149     | [], [] -> []
150     | xhd :: xtl, yhd :: ytl ->
151       xhd * yhd :: prods xtl ytl
152     | _, _ -> failwith "prods: lists of different length"
153
154 .....
155 Exercise 4: Rewrite the recursive `prods` function as `prods_iter`
156 using a `while` loop. Don't forget to handle the case where the two
157 lists have different lengths, by raising an appropriate exception.
158 .....*)
159
160 let prods_iter (xs : int list) (ys : int list) : int list =

```

```

161   failwith "prods_iter not implemented" ;;
162
163
164   (*=====
165   Part 3: Tail recursion as a solution to stack space problems
166
167   In a tail-recursive function the recursive invocation *is* the final
168   result of the invoking call. The value of the recursive function is
169   immediately returned; no further computation must be done to it, so no
170   suspended computation needs to be stored on the call stack, thus
171   avoiding the problem of stack overflow.
172
173   Below, the `length` function from above has been rewritten to use a
174   tail-recursive auxiliary function `length_tr` (the "tr" stands for
175   "tail-recursive") to demonstrate this:
176
177       let length lst =
178         let rec length_tr lst accum =
179           match lst with
180             | [] -> accum
181             | _hd :: tl -> length_tr tl (1 + accum) in
182         length_tr lst 0 ;;
183
184   The technique used here, using a tail-recursive auxiliary function
185   that makes use of an added argument that acts as an accumulator for
186   the partial results, is a common one for converting functions to
187   tail-recursive form.
188
189   .....
190   Exercise 5: Tail-recursive sum
191
192   Rewrite the `sum` function from above to be tail recursive. (As usual,
193   for this and succeeding exercises, you shouldn't feel beholden to how
194   the definition is introduced in the skeleton code below. For instance,
195   you might want to add a `rec`, or use a different argument list, or no
196   argument list at all but binding to an anonymous function instead.)
197
198   Verify your implementation is tail-recursive by summing up the
199   elements of an extremely long list, like this 1,000,000 element list:
200
201       # sum (List.init 1_000_000 Fun.id) ;;
202       - : int = 499999500000
203
204   Gauss would be proud!
205   .....*)
206
207   let sum _ =
208     failwith "sum not implemented" ;;
209
210   (*.....
211   Exercise 6: Write a tail-recursive version of the `prods` function
212   from Exercise 3, which finds the product of the corresponding elements
213   of two integer lists.
214

```

```

215 Your initial try at a tail-recursive function may output a list that
216 is in reverse order of your expected output. This is a common outcome
217 in tail-recursive list iteration functions. (In general, you'd want to
218 consider whether or not this has negative outcomes on your intended
219 use case. It may be that the output order is not significant.)
220
221 In this case, for testing purposes, please preserve the initial
222 ordering of the lists. One method to do so is simply to reverse the
223 list at the end, using a tail-recursive reversal function of
224 course. Fortunately, you implemented such a function in Exercise 2. In
225 addition, the built-in `List.rev` is tail-recursive (even though the
226 documentation doesn't mention that fact).
227 .....*)
228
229 let prods _ =
230   failwith "prods not implemented" ;;
231
232 (* You've now implemented `prods` a few times, so think about which of
233 them you think is the most time efficient, which the most space
234 efficient, and which required the most work to write. Remember the
235 famous quotation from computer scientist Donald Knuth: "We should
236 forget about small efficiencies, say about 97% of the time: premature
237 optimization is the root of all evil."
238
239 Tail recursion is a type of optimization, and it may not always be
240 worth the sacrifice in development time and code readability.
241 Iterative solutions in a functional programming language like OCaml
242 may also not be worth the time. It is critical to assess the impact
243 that your use cases will have on your design. *)
244
245 (* .....
246 Exercise 7: Modify your `prods` function to use option types to deal
247 with lists of different lengths. Call it `prods_opt`. The `prods_opt`
248 function should return `None` if its two list arguments are of
249 different lengths, and if the lists are of the same length, `Some lst`
250 where `lst` is the products of the corresponding elements of its
251 arguments.
252 .....*)
253
254 let prods_opt (xs : int list) (ys : int list) : int list option =
255   failwith "prods not implemented" ;;
256
257 (* .....
258 Exercise 8: Finally, combine your `sum` and `prods` functions to
259 create a tail-recursive dot product function (that is, the sum of the
260 products of corresponding elements of the lists). (For reference, you
261 implemented dot product in lab 2.)
262 .....*)
263
264 let dotprod _ =
265   failwith "dotprod not implemented" ;;
266
267
268 (*=====

```

```

269 Part 4: The Mario problem
270
271 As you've observed in this lab, procedural programming can be useful,
272 but most problems can and typically should be solved with functional
273 techniques. However, there is one famous problem that you may be
274 familiar with a procedural implementation of: CS50's Mario. (For those
275 unfamiliar, the task is to print a half-pyramid of lines of a given
276 height as shown in the example below. See
277 https://docs.cs50.net/2017/ap/problems/mario/less/mario.html.) *)
278
279 (*.....
280 Exercise 9: Implement a function in the procedural paradigm that
281 prints out a half-pyramid of a specified height, per the below. Use
282 hashes (#) for blocks. The function should raise an `Invalid_argument`
283 exception for heights greater than 23. (Why? I don't know. That's just
284 how CS50 did it.)
285
286     # mario 5 ;;
287         ##
288         ###
289         ####
290         #####
291         ######
292     - : unit = ()
293
294     # mario 24
295     Exception: Invalid_argument "This pyramid is way too high for Mario".
296
297 .....*)
298
299 let mario (height : int) : unit =
300     failwith "mario not implemented" ;;

```